Riassunto modulo 2

From Inverse Problems to Neural Imaging Pipelines

Nel primo modulo del corso, il problema della **ricostruzione di immagini** è stato formulato come un **problema inverso**. In forma discreta, esso si esprime come:

$$y^\delta = Kx^* + e$$

dove:

- $K \in \mathbb{R}^{m \times n}$ è l'**operatore di acquisizione discreto** (ad esempio una trasformata di Fourier, un operatore di blur, una proiezione tomografica, ...)
- x^* è l'**immagine incognita** che si vuole ricostruire
- y^δ rappresenta i **dati osservati**
- e modella il **rumore** e le **perturbazioni** del sistema di misura

I metodi classici di ricostruzione non tentano di invertire direttamente K (operazione spesso instabile o mal posto), ma calcolano una stima di x risolvendo un problema variazionale del tipo:

$$\hat{x} = \arg\min_x (\mathcal{D}(Kx, y^\delta) + \mathcal{R}(x))$$

dove:

- $\mathcal{D}(Ax, y)$ è un **termine di data fidelity**, che misura quanto la soluzione è coerente con i dati (ad esempio una norma)

  → da un lato si vuole rispettare i dati acquisiti
- $\mathcal{R}(x)$ è un **termine di regolarizzazione**, che incorpora conoscenza a priori sull'immagine (es. sparsità, smoothness, Total Variation)

  → dall'altro si impongono vincoli per rendere il problema ben posto (stabilità, unicità della soluzione)

Con l'introduzione delle reti neurali, questa struttura matematica **non viene abbandonata**, ma piuttosto **riformulata** in un linguaggio più adatto alla computazione in tensori, all'ottimizzazione stocastica e a grandi database

In una strategia di ricostruzione **end-to-end**, si addestra un modello f_Θ che mappa direttamente il dato misurato y in un'immagine ricostruita $\hat{x}$

$$\hat{x} = f_\Theta(y^\delta)$$

Questo approccio è particolarmente attraente dal punto di vista computazionale: una volta completata la fase di training, l'inferenza è molto veloce, poiché una **singola forward pass** della rete può sostituire un lungo algoritmo iterativo

In un approccio **ibrido**, si adotta una strategia diversa rispetto al caso end-to-end: si mantiene un **passo esplicito di ricostruzione**, basato sulla fisica del problema o su un risolutore classico, e si utilizza la rete neurale solo come **componente di un metodo più ampio**

Questo approccio è generalmente **più costoso dal punto di vista computazionale**, poiché richiede più iterazioni e combina moduli diversi (fisici e appresi).

Tuttavia, offre un vantaggio importante: consente un **maggiore controllo sulla data consistency** e sulla relazione tra la ricostruzione $\hat{x}$ e il modello di misura

Representing and Preparing Images for Neural Networks

Una immagine in scala di grigi viene rappresentata come un tensore di forma

$$(C, H, W)$$

con C=1, mentre un minibatch di B immagini in scala di grigi è rappresentato come

$$(B, C, H, W)$$

Per immagini RGB si ha C=3. Nel caso di immagini multispettrali o dati fisicamente più ricchi, la dimensione dei **canali** può rappresentare ulteriori informazioni

> ❗ Nelle pipeline di *neural imaging*, le convenzioni sulla forma (*shape*) dei tensori sono parte integrante del modello matematico. Un tensore di forma (H, W) e un tensore di forma (1, 1, H, W) possono rappresentare visivamente la stessa immagine, ma non svolgono lo stesso ruolo all'interno di una rete neurale e possono generare errori

Le immagini memorizzate come `uint8` assumono tipicamente valori in [0, 255]. Le reti neurali raramente vengono addestrate direttamente su questa scala → invece, si effettua una riscalatura dei dati, ad esempio in [0, 1] in modo che **attivazioni e gradienti** rimangano in intervalli numericamente stabili

→ la **scala dell'input** influisce direttamente sulla **scala della loss**, e quindi modifica la **dinamica dell'ottimizzazione**

Un secondo aspetto riguarda il **tipo di dato.** Nella maggior parte delle applicazioni di imaging, il formato standard è **float32** che rappresenta un buon compromesso tra **uso della memoria** e **precisione numerica**

- precisioni più basse → più veloci ma potenzialmente troppo approssimate per problemi inversi delicati
- precisioni più alte → utili per debugging, verifica degli operatori o pipeline instabili, ma con costo elevato in memoria

Datasets, DataLoaders, and a Mayo-Style Example

Una volta che le immagini sono memorizzate in un formato compatibile con i tensori, è ancora necessario un meccanismo per fornirle in modo efficiente durante l'addestramento: in PyTorch, questo è il ruolo delle astrazioni `Dataset` e `DataLoader`

> → poiché le immagini mediche e i tensori associati occupano molta memoria, non possiamo caricare l'intero dataset contemporaneamente sulla GPU: queste due classi lavorano in sinergia proprio per gestire il flusso di dati in modo ottimizzato
>
> - Un **Dataset** definisce come viene recuperato un singolo campione
> - Un **DataLoader** a sua volta "avvolge" il dataset e produce *minibatch* mescolati

🧠 Consideriamo un esempio che utilizzeremo più volte nel corso: il *Mayo's dataset*
→ il dataset Mayo è comunemente utilizzato per testare e valutare (*benchmarking*) gli algoritmi di ricostruzione di immagini per Tomografia Computerizzata (TC) a partire da misurazioni sparse (*sparse measurement*)
Lavorare con "misurazioni sparse" significa cercare di ricostruire l'interno del corpo avendo a disposizione meno "scatti" (proiezioni angolari) rispetto a una TAC convenzionale
→ questo è l'obiettivo della moderna TC a basso dosaggio: ridurre le radiazioni potenzialmente dannose assorbite dal paziente
→ tuttavia, avere meno misurazioni rende il problema inverso fortemente mal posto, poiché i dati scarseggiano, richiedendo reti neurali e algoritmi di ottimizzazione avanzati per compensare la mancanza di informazioni ed evitare artefatti nell'immagine ricostruita
[Mayo's Dataset Download]
Questo dataset comprende un totale di N=3305 immagini di addestramento e N=327 immagini di test, che rappresentano le scansioni TC del torace di 10 individui

PyTorch, Automatic Differentiation, and Training Mechanics

Computational Representation and Differentiation

In **PyTorch**, l'oggetto di base è il `tensor`. Da un punto di vista matematico, un tensore è semplicemente un array multidimensionale, ma in pratica porta con sé informazioni aggiuntive:

- **shape (forma)**, che ci indica le dimensioni;
- **data type (tipo di dato)**, che determina la precisione numerica;
- **device (dispositivo)**, che ci dice se il tensore risiede su CPU o GPU;
- **gradient tracking (tracciamento del gradiente)**, che ci indica se devono essere registrate le derivate rispetto a quel tensore.

Un'immagine in scala di grigi di dimensioni $H \times W$ è rappresentata come un tensore di forma (H, W), ma in una pipeline di addestramento è quasi sempre incorporata nel formato standard:

$$(B, C, H, W)$$

Il tipo di dato (*data type*) descrive come i numeri sono rappresentati in memoria, definendo esplicitamente un compromesso tra consumo di memoria e precisione algoritmica.

Nei sistemi moderni, esistono 4 rappresentazioni di sistemi numerici principalmente utilizzate:

- **float64**: solitamente chiamato **doppia precisione**. Alloca un totale di 11 cifre (binarie) per l'esponente, 52 cifre (binarie) per la mantissa e 1 cifra (binaria) per il segno. È il modo più preciso di rappresentare un numero, ma è raramente utilizzato nell'IA a causa della grande quantità di spazio richiesta per rappresentare ogni numero (8 byte per numero).
- **float32**: solitamente chiamato **singola precisione**. È il `dtype` predefinito per i tensori di PyTorch, poiché richiede solo 4 byte per numero mantenendo un alto livello di precisione.
- **float16**: solitamente chiamato **mezza precisione**. Richiede solo 2 byte per numero, rendendolo estremamente efficiente ma anche non molto preciso. È diventato popolare di recente in quanto comunemente usato per implementare modelli molto grandi come gli LLM. È di scarso utilizzo nelle configurazioni di imaging poiché solitamente richiedono maggiore precisione.

- **bfloat16**: una variante della rappresentazione a mezza precisione, che alloca meno cifre all'esponente e più alla mantissa. Il risultato è una precisione maggiore rispetto a `float16`, ma allo stesso tempo una maggiore probabilità di overflow.

Una rete neurale è costruita componendo dei moduli. Ogni modulo corrisponde a una mappa parametrizzata. Per esempio:

- **Linear** implementa la trasformazione lineare $\boldsymbol{z} \mapsto W\boldsymbol{z} + \boldsymbol{b}$;
- **Conv2d** implementa una **convoluzione** discreta con kernel apprendibili (dettagli sulla convoluzione e sulle reti neurali convoluzionali saranno forniti nelle sezioni seguenti);
- le **funzioni di attivazione** implementano mappe non lineari puntuali;
- i **livelli di normalizzazione** modificano le statistiche delle feature intermedie.

Computational Graphs and Automatic Differentiation

Mentre elabora il calcolo della loss $L(\boldsymbol{\Theta})$, **PyTorch** registra il **grafo computazionale**, un grafo aciclico diretto i cui nodi sono operazioni e i cui archi trasportano i tensori intermedi. Quando necessario, questo grafo ci permette di calcolare

$$\frac{\partial L(\boldsymbol{\Theta})}{\partial \boldsymbol{\Theta}}$$

attraverso la **differenziazione automatica** percorrendo il grafo a ritroso.

Da un punto di vista matematico, si tratta semplicemente della regola della catena (*chain rule*) applicata sistematicamente. L'importanza di **PyTorch** è che automatizza questo processo in modo efficiente anche quando la rete contiene milioni di parametri, e anche quando il grafo computazionale mescola livelli appresi con operazioni tensoriali arbitrarie.

Training Mechanics in Practice

Nel codice, una loss è un tensore scalare (ovvero un tensore di forma `(1, 1, 1, 1)`). Una volta calcolata, si chiama il metodo `.backward()` per ottenere tutti i gradienti dei parametri richiesti. Successivamente, un **ottimizzatore** aggiorna i parametri stessi.

Per semplificare l'implementazione, il sottomodulo `optim` di PyTorch contiene un'ampia scelta di diversi ottimizzatori predefiniti, come `SGD`, `Adam`, `AdamW` e altri. Ogni ottimizzatore prende come input l'insieme dei parametri che devono essere ottimizzati (i parametri di un dato `Module` sono accessibili tramite il metodo `.parameters()`) e un tasso di apprendimento (*learning rate*, che rappresenta il termine η). Successivamente, l'addestramento procede come segue:

La maggior parte delle volte, impostare un tasso di apprendimento (*learning rate*) fisso e invariante non è una buona pratica, poiché può portare a situazioni come quella descritta in figura: se η viene scelto troppo alto, i parametri potrebbero iniziare a rimbalzare avanti e indietro attorno al minimo, dando vita a un modello che non migliora le proprie prestazioni durante l'addestramento.

Per evitare questo tipo di problemi, in PyTorch è possibile definire un **learning rate schedule**: una funzione predefinita che consente di cambiare il valore del tasso di apprendimento mentre l'addestramento procede. All'interno del suo sottomodulo `optim`, PyTorch implementa molti diversi scheduler del tasso di apprendimento.

Tra questi, i più rappresentativi sono:

- **ReduceLROnPlateau**: Riduce il tasso di apprendimento di una quantità predefinita ogni volta che la funzione di loss smette di diminuire.
- **LinearLR**: Riduce linearmente il tasso di apprendimento in un dominio dato.

- **CosineAnnealingLR**: Lo scheduler del tasso di apprendimento più comune. Riduce gradualmente il tasso di apprendimento fino a zero seguendo una trasformazione cosinusoidale.

From PyTorch to `IPPy`

`IPPy` è progettata precisamente per questo scopo. Non sostituisce PyTorch: lo estende con componenti orientati all'imaging che si comportano come normali moduli differenziabili all'interno dello stesso grafo computazionale. In particolare:

- **operators** contiene modelli in avanti come sfocamento (*blurring*), riduzione di scala (*downscaling*), operatori gradiente e proiettori CT;
- **solvers** contiene algoritmi di ricostruzione classici, utili come baseline o blocchi costruttivi;
- **nn** contiene architetture neurali per problemi inversi.

Why This Computational Viewpoint Matters for Imaging

- La **precisione finita** influisce sulla stabilità numerica. L'aritmetica standard a singola precisione `float32` può accumulare errori di arrotondamento attraverso molti livelli. Nei problemi inversi mal condizionati, dove piccole perturbazioni nell'input possono amplificarsi enormemente, questo non è trascurabile, e la differenza tra `float32` e `float64` può essere visibile nell'immagine ricostruita.
- La **normalizzazione** modifica la scala e le proprietà statistiche delle rappresentazioni intermedie. La *Batch Normalization* normalizza utilizzando statistiche calcolate sull'intero *minibatch* corrente, il che accoppia elementi elaborati insieme e introduce una dipendenza dalla composizione del batch. La *Layer Normalization* opera su ogni campione in modo indipendente, rendendola più adatta a scenari in cui la dimensione del batch è molto piccola, situazione comune nell'imaging.
- L'**aumento dei dati (Data Augmentation)** modifica la distribuzione empirica osservata durante l'addestramento. Una rete addestrata su immagini che sono state specchiate orizzontalmente in modo casuale imparerà implicitamente una regola di ricostruzione invariante alla riflessione destra-sinistra. Se questa simmetria sia fisicamente appropriata dipende interamente dalla geometria dell'imaging. L'aumento dei dati non è solo un trucco di regolarizzazione; è una codifica implicita di conoscenza a priori sul problema.

From Machine Learning to Neural Networks

Basics of Machine Learning

Un problema di apprendimento è definito da un dataset di N coppie input-output:

$$\mathcal{D} = \{(\boldsymbol{x}^{(i)}, \boldsymbol{y}^{(i)})\}_{i=1}^{N}$$

dove $\boldsymbol{x}^{(i)} \in \mathbb{R}^d$ è l'input e $\boldsymbol{y}^{(i)} \in \mathbb{R}^s$ è l'output target. L'obiettivo è imparare una funzione parametrizzata $f_\Theta : \mathbb{R}^d \to \mathbb{R}^s$ che mappi correttamente gli input negli output al di fuori del set di addestramento.

Fondamentalmente, un modello di ML è una funzione $f_{\boldsymbol{\Theta}} : \mathbb{R}^d \to \mathbb{R}^s$ il cui comportamento è interamente determinato dal suo vettore di parametri $\boldsymbol{\Theta}$. La scelta di s dipende dal compito: in un problema di regressione s è il numero di valori previsti; nella ricostruzione di immagini s è il numero di pixel.

Per costruire un'intuizione, iniziamo con il caso più semplice $d = s = 1$:

$$f_{\boldsymbol{\Theta}}(x) = \Theta_1 + \Theta_2 x$$

- Diverse scelte di $\boldsymbol{\Theta}$ portano a previsioni *molto diverse*.

- Dato un set di addestramento, alcune scelte di parametri sono migliori di altre, e il processo di ricerca della scelta migliore è chiamato **addestramento** (*training*).

From Linear Predictors to Neural Networks

L'idea di base dietro le reti neurali è piuttosto semplice: i modelli lineari sono efficienti ma non abbastanza espressivi, quindi perché non impilare più modelli lineari l'uno sull'altro per migliorare l'espressività?

→ tuttavia, impilare mappe lineari senza ulteriori operazioni non lineari non amplia la classe del modello in alcun modo essenziale ⇒ la profondità conta, ma solo una volta che la linearità è stata interrotta.

Come vedremo in seguito, versioni più avanzate di reti neurali possono essere ottenute:

- Impilando più modelli lineari (inserendo sempre una funzione non lineare tra di essi),
- Cambiando la funzione non lineare utilizzata,
- Limitando la struttura delle matrici dei parametri W^l per consentire un calcolo più efficiente (questo è particolarmente importante per la ricostruzione delle immagini).

La scelta dell'attivazione è una decisione di modellazione con conseguenze reali. Le attivazioni più comunemente incontrate nelle architetture di imaging sono elencate qui (includiamo anche la ReLU per completezza):

- **ReLU**, $\rho(t) = \max\{t, 0\}$, è la scelta predefinita per le reti convoluzionali. È computazionalmente economica, evita il problema della scomparsa del gradiente (*vanishing gradient*) che affliggeva le precedenti architetture basate sulla sigmoide, e produce attivazioni sparse. La sua debolezza principale è che i neuroni che ricevono input costantemente negativi smettono interamente di contribuire (**dead neurons**), il che può rallentare o bloccare l'addestramento.
- **Leaky ReLU**, $\rho(t) = \max\{\alpha t, t\}$ con un piccolo $\alpha > 0$, è un semplice rimedio: una piccola pendenza non nulla sulla semiretta negativa assicura che i gradienti possano sempre fluire all'indietro, prevenendo il problema dei neuroni morti e preservando la maggior parte dei vantaggi computazionali della ReLU.
- **Sigmoide**, $\sigma(t) = \frac{e^t}{1+e^t}$, e **Tanh**, $\tanh(t) = \frac{e^t - e^{-t}}{e^t + e^{-t}}$, sono tra le attivazioni più vecchie. Entrambe saturano per valori elevati di $|t|$, causando la scomparsa dei gradienti nelle reti profonde; per questo motivo sono state ampiamente sostituite dalla ReLU nei livelli nascosti. Rimangono rilevanti come non linearità di output e in architetture ricorrenti a gate come le LSTM.
- **GELU** è un'attivazione liscia e differenziabile motivata dalla regolarizzazione stocastica. È la scelta standard nelle architetture basate su transformer, dove la sua levigatezza interagisce meglio con il meccanismo di self-attention rispetto alla soglia netta della ReLU.
- **SiLU** (Sigmoid Linear Unit), $\rho(t) = t \cdot \sigma(t)$, è un'attivazione liscia a gate automatico (*self-gated*) strettamente correlata alla GELU e utilizzata in EfficientNet e in diverse altre moderne architetture convoluzionali.
- **Softmax**, $\operatorname{softmax}(\boldsymbol{z})_i = e^{z_i} / \sum_j e^{z_j}$, mappa un vettore a valori reali in una distribuzione di probabilità. Non viene utilizzata come attivazione per i livelli nascosti, ma appare come non linearità dell'ultimo livello nei compiti di classificazione e, nella segmentazione delle immagini, converte la mappa delle feature in uscita in probabilità di classe per ogni pixel.

Ogni rete neurale può essere rappresentata come un grafo a livelli in cui ogni nodo è chiamato **neurone**. I livelli sono:

- **Input layer** (d neuroni): contiene direttamente l'input $\boldsymbol{x}$.
- **Hidden layer(s)** (H neuroni ciascuno, con H che rappresenta la **hidden dimension**): rappresentazioni intermedie non direttamente osservabili dall'esterno. L'i-esimo neurone in un livello nascosto calcola $z_i = \boldsymbol{w}_i^T \boldsymbol{x} + b_i$, dove $\boldsymbol{w}_i$ e b_i sono i **pesi** e il **bias** specifici del neurone, rappresentati come gli archi che collegano il livello precedente a z_i. La funzione di attivazione mappa poi $z_i \mapsto h_i = \rho(z_i)$.
- **Output layer** (s neuroni): produce la previsione finale $\boldsymbol{y}$.

I vettori $\boldsymbol{h}_\ell$ sono le feature interne prodotte dalla rete. Nei primi livelli si trovano tipicamente rilevatori di eventi locali semplici: bordi orientati, confini di contrasto e texture ripetute di basso livello. In un contesto di imaging, queste sono esattamente le strutture elementari che gli artefatti di acquisizione e il rumore distorcono in modo più evidente.

Nei livelli intermedi, le feature elementari vengono combinate in strutture su scala più ampia (in un'immagine medica, ciò potrebbe significare i confini di un organo o pattern geometrici caratteristici della geometria di imaging). Nei livelli più profondi, le rappresentazioni diventano specifiche per il compito (*task-specific*): codificano la differenza tra una vera struttura anatomica e un artefatto di ricostruzione, o tra un bordo autentico e il *ringing* introdotto dal troncamento della misurazione.

Optimization, Backpropagation, and Generalization

Addestrare una rete neurale significa trovare i parametri $\boldsymbol{\Theta}$ tali che la previsione del modello $f_{\boldsymbol{\Theta}}(\boldsymbol{x}^{(i)})$ approssimi al meglio il target reale $\boldsymbol{y}^{(i)}$ sull'intero set di addestramento. In pratica, ciò richiede la risoluzione del problema di ottimizzazione non convessa della funzione di Loss

Questo problema viene solitamente risolto tramite la discesa del gradiente (*gradient descent*), ovvero iterando

Tuttavia, poiché nelle applicazioni pratiche il numero totale di punti dati N può essere molto grande, il calcolo può essere computazionalmente proibitivo. Per questo motivo, in pratica si calcola raramente il gradiente empirico completo. Si utilizzano invece i **minibatch** e si ottiene un'approssimazione stocastica → algoritmo risultante: **Discesa del Gradiente Stocastica (SGD)**

Un altro concetto fondamentale da spiegare esplicitamente è la **generalizzazione**. Una rete può minimizzare molto bene la *loss* di addestramento eppure fallire su nuovi dati. Nel contesto dei problemi inversi, questa questione è ancora più delicata perché il dato di test può differire da quello di addestramento non solo nel contenuto dell'immagine, ma anche per:

- **livello di rumore**;
- **geometria di acquisizione**;
- **calibrazione**;
- **discretizzazione**;
- **classe anatomica** o classe dell'oggetto.

Convolutions, CNNs, and End-to-End Training

Convolutions and Padding

Possiamo distinguere tre tipi di informazioni:

- **Informazione puntuale:** qualsiasi informazione che può essere estratta considerando un singolo pixel isolato dal suo contesto.
- **Informazione locale:** qualsiasi informazione estratta da un piccolo insieme di pixel vicini.
- **Informazione globale:** qualsiasi informazione che richiede la conoscenza dell'intera immagine.

I compiti di elaborazione delle immagini si basano principalmente su informazioni locali e globali, con solo una piccolissima quantità di informazioni puntuali.

Da questa osservazione si può notare che la classica architettura MLP, definita da una catena di trasformazioni lineari alternate a funzioni di attivazione non lineari, non è una scelta naturale per le immagini perché elabora ogni pixel come un singolo neurone isolato. È invece preferibile considerare una rete in cui ogni livello **lineare** sia sostituito da un livello che tenga conto dell'**informazione locale**. Lo strumento matematico più semplice che permette di ottenere questo risultato è l'operazione di **convoluzione**.

Una convoluzione è un'operazione lineare che coinvolge un'immagine di input $\boldsymbol{x}$ con forma $n_x \times n_y$ e un kernel di convoluzione $\mathcal{K}$ con forma $k \times k$, solitamente molto più piccolo dell'immagine stessa. Ogni pixel nella posizione (i, j) dell'immagine di output $\boldsymbol{y} = \mathcal{K} * \boldsymbol{x}$ si ottiene centrando il kernel sul pixel (i, j) dell'immagine di input, moltiplicando il kernel elemento per elemento per i corrispondenti pixel di input e sommando il risultato.

Di conseguenza, ogni pixel dell'immagine di output $\boldsymbol{y}$ è influenzato da un intorno $k \times k$ dell'immagine di input $\boldsymbol{x}$, imponendo la necessaria località dell'operazione.

Un limite chiave delle convoluzioni, chiaramente visibile nell'immagine sopra, è che l'immagine di output ha una dimensione molto più ridotta rispetto al corrispondente input. Questo rende le convoluzioni semplici poco pratiche per i compiti di ricostruzione d'immagine *end-to-end*, poiché solitamente ci aspettiamo che la forma dell'output corrisponda a quella dell'input.

Per superare questa limitazione, una tecnica comune è il **padding**, o **convoluzione con padding**. L'idea è quella di circondare l'immagine di input con un numero adeguato di zeri prima di applicare la convoluzione, in modo da preservare la forma dell'output dopo l'applicazione del kernel. In pratica, se il kernel ha forma $k \times k$, si dovrebbe applicare all'input un padding di $\frac{k-1}{2}$ zeri su ogni lato per preservare la forma dell'immagine. Il risultato di una convoluzione con padding è mostrato nell'immagine seguente.

Convolutional Neural Networks (CNN)

Sfruttando l'osservazione precedente, possiamo progettare un'architettura di rete neurale specifica per lavorare con le immagini. La sua versione più semplice, chiamata **Rete Neurale Convoluzionale (CNN)**, si ottiene prendendo un classico Percezione Multistrato (ovvero l'architettura di rete neurale più semplice descritta nel capitolo precedente di questo corso) e sostituendo il livello `Linear` con un livello `Conv2d`. I **parametri** di una CNN sono i numeri contenuti nei kernel di convoluzione, che determinano il modo in cui i pixel vengono elaborati.

In generale, ogni livello convoluzionale è composto da un ampio insieme predefinito di kernel di convoluzione della stessa forma; ognuno di essi elabora la stessa immagine in parallelo, estraendo da essa informazioni differenti. Una pratica comune consiste nell'impilare più livelli convoluzionali uno dopo l'altro, alternandoli con funzioni di attivazione non lineari come la **ReLU.**

Il ciclo di addestramento è estremamente semplice. Ad ogni iterazione:

1. carichiamo un batch di immagini pulite x dal dataset Mayo;
2. generiamo le corrispondenti misurazioni sfocate e rumorose $y^\delta = Kx + e$;
3. passiamo y^δ alla rete;
4. confrontiamo l'output con il batch pulito x tramite la `MSELoss`;
5. aggiorniamo i parametri con la backpropagation.

Questo è il meccanismo standard di addestramento supervisionato per i problemi inversi sintetici: la rete riceve dati corrotti come input e immagini pulite come target, mentre le corruzioni vengono generate "online" a partire dal modello forward noto.

Le CNN sono un'ottima alternativa agli MLP perché elaborano i dati basandosi sull'informazione locale invece che su quella puntuale, il che si adatta meglio alla nostra intuizione su come dovrebbero essere trattate le immagini.

Mentre abbiamo già osservato che la scelta predefinita per la maggior parte delle funzioni di attivazione è la ReLU o una sua variante, è necessario prestare particolare attenzione quando si sceglie la funzione di attivazione per l'**ultimo livello**.

> Possiamo considerare altre funzioni di attivazione che potrebbero essere più adatte per i compiti di elaborazione delle immagini, ciascuna con i propri pro e contro:
>
> - **ReLU:** L'uso della ReLU come attivazione finale sembra promettente: poiché sappiamo già che ogni immagine standardizzata dovrebbe trovarsi nell'intervallo [0, 1], imporre pixel non negativi sembra naturale. Infatti, quando si usa la ReLU come attivazione finale, l'immagine ricostruita appare spesso migliore, in particolare

nello sfondo, dove diventa più facile per il modello generare una regione perfettamente nera. Tuttavia, la ReLU ha lo svantaggio di non essere differenziabile in 0, che è esattamente il valore che vorremmo raggiungere nello sfondo, e può quindi causare instabilità nell'addestramento.

- **Sigmoide:** Dal punto di vista dell'intervallo (*range*), la sigmoide sembra ancora più promettente, poiché il suo range è (0, 1), che è quasi lo stesso di un tensore d'immagine standardizzato. Sebbene questa scelta garantisca che la ricostruzione si trovi nel dominio corretto, l'addestramento con questa attivazione finale è solitamente piuttosto instabile. Questo perché la sigmoide **tende a** 0 e 1 rispettivamente a $-\infty$ e $+\infty$. Pertanto, quando si cerca di far corrispondere il valore 0 nello sfondo, l'ottimizzatore tende a spingere i pesi verso l'infinito, causando instabilità.
- **Tanh:** La Tanh è la funzione di attivazione più comune quando si lavora con le CNN Residuali (che introdurremo in seguito) perché il suo range è (-1, 1), il che la rende adatta per apprendere un'approssimazione della mappatura residua. Quando viene usata su una CNN non residua, tuttavia, questa attivazione finale soffre di problemi di instabilità simili a quelli della sigmoide.

Residual Learning, Receptive Field, and UNet

Residual CNN

Qualsiasi CNN (Rete Neurale Convoluzionale) per l'elaborazione delle immagini funziona identificando *pattern* (schemi visivi ricorrenti) nell'immagine di input, che sono stati appresi in precedenza durante l'addestramento, e ricostruendo ogni pattern nello stesso modo in cui è stato appreso. Per questo motivo, se, ad esempio, addestriamo un modello per rimuovere la sfocatura (*deblur*) da immagini di gatti, non c'è alcuna garanzia che lo stesso modello sarà in grado di rimuovere la sfocatura da immagini di cani: i diversi pattern presenti nelle immagini dei cani rendono la ricostruzione accurata molto più difficile.

Di conseguenza, un vasto filone di ricerca nell'elaborazione delle immagini basata su CNN cerca di insegnare ai modelli a identificare i pattern che sono tipici del *task* (il problema specifico) che si vuole risolvere, piuttosto che i pattern specifici dei dati di addestramento. A tal fine, possiamo osservare che mentre i pattern nell'immagine sono di solito specifici del dataset, l'*artefatto* (ovvero la differenza tra l'immagine corrotta e l'immagine vera) tende a mostrare pattern strettamente legati al task.

Ad esempio, consideriamo il seguente problema, in cui due immagini provenienti da due dataset completamente diversi sono corrotte da sfocatura Gaussiana e rumore. Guardando sia l'immagine vera che quella corrotta, è chiaro che il *contenuto* dei due esempi è profondamente diverso. D'altra parte, quando guardiamo al *residuo*, ovvero il valore della differenza tra dato degradato e dato pulito $|x_{true} - y^{\delta}|$, i pattern nelle due immagini appaiono estremamente simili.

Questa osservazione giustifica il fatto che *apprendere il residuo è solitamente più semplice (e generalizza meglio) rispetto ad apprendere l'immagine vera e propria.*

L'operazione di sommare l'input all'output con l'obiettivo di apprendere il residuo è solitamente chiamata *residual connection* (connessione residua) o *skip connection* (connessione di salto). Queste architetture prendono il nome di Reti Neurali Convoluzionali Residue (ResCNN).

Beyond CNN: the Receptive field

Le moderne architetture di reti neurali per i task di elaborazione delle immagini raramente utilizzano il semplice modello CNN descritto in precedenza. Un motivo è il concetto di **Campo Recettivo** (*Receptive Field*, RF).

Per comprenderlo, consideriamo un modello con L layer convoluzionali, ciascuno con dimensione del kernel $k_l = 3$, come illustrato nella figura seguente. È evidente che ogni pixel nella *feature map* (mappa delle caratteristiche) del secondo layer è influenzato solo da una porzione 3×3 del primo layer, ovvero l'input della rete. Allo stesso modo, ogni

pixel nella *feature map* del terzo layer è influenzato da una porzione 3×3 del layer precedente e, di conseguenza, da una porzione 5×5 dell'immagine di input.

Il numero di pixel dell'immagine di input che influenzano il valore di ciascun pixel nella i-esima *feature map* è ciò che chiamiamo il **campo recettivo** dell'i-esimo layer. Continuando questo ragionamento attraverso la rete fino all'output, possiamo calcolare il campo recettivo dell'intera rete, che rappresenta il numero totale di pixel dell'immagine di input che influenzano il singolo pixel di output della rete.

Questa quantità è cruciale perché, quando si ricostruiscono dati corrotti contenenti artefatti, comprendere il Campo Recettivo aiuta a garantire che il modello catturi e gestisca tali artefatti in modo accurato. In particolare, quando gli artefatti sono locali, un piccolo campo recettivo è sufficiente per dare al modello la capacità di distinguere tra l'artefatto e le caratteristiche dell'immagine; al contrario, quando gli artefatti sono globali, è necessario un RF molto ampio per produrre una ricostruzione accurata.

UNet

La U-Net è una variante multi-scala dell'architettura delle Reti Neurali Convoluzionali, ed è in grado di rilevare e correggere efficacemente sia gli artefatti globali che quelli locali. La sua idea centrale è semplice ma estremamente potente: invece di elaborare l'immagine a una singola risoluzione, la rete la analizza attraverso una gerarchia di scale e poi ricombina le informazioni durante la ricostruzione dell'output.

Ad alto livello, una U-Net è un'architettura *encoder-decoder* completamente convoluzionale (*fully convolutional*). L'encoder riduce progressivamente la risoluzione spaziale attraverso operazioni di sottocampionamento (*downsampling*), aumentando simultaneamente il numero di canali delle *feature* (caratteristiche). Ciò rende la rappresentazione più compatta e allarga rapidamente il campo recettivo. Il decoder inverte poi il processo, ripristinando progressivamente la risoluzione spaziale finché l'output non ha la stessa dimensione dell'input.

Più formalmente, l'encoder divide naturalmente la rete in distinti livelli di risoluzione, che denotiamo con g, con $g = 0, \dots, \mathcal{G}$, dove $\mathcal{G} + 1$ è il numero totale di livelli nella rete. Ad ogni livello, il numero di canali viene solitamente aumentato man mano che la risoluzione diminuisce. Se c_0 denota il numero di canali nel primo livello, una scelta molto comune è:

$$c_{g+1} = 2c_g, \quad g = 0, \dots, \mathcal{G} - 1$$

Questo è uno dei motivi per cui le U-Net rimangono gestibili dal punto di vista computazionale: i layer più profondi hanno molti canali, ma operano su griglie spaziali molto più piccole.

La caratteristica più distintiva della U-Net è la presenza di **skip connections tra i livelli dell'encoder e del decoder**. Quando le mappe delle caratteristiche (*feature maps*) vengono sottocampionate, alcune informazioni spaziali ad alta frequenza vengono inevitabilmente compresse. Le skip connections trasferiscono le mappe delle caratteristiche dall'encoder direttamente al decoder allo stesso livello di risoluzione, permettendo al percorso di ricostruzione di recuperare dettagli fini che altrimenti sarebbero difficili da ricostruire. In pratica, queste skip connections sono uno dei motivi principali del successo dell'architettura nella segmentazione, nel *denoising*, nel *deblurring*, in tomografia e in molti altri task *image-to-image*.

Una seconda scelta di progettazione importante riguarda il **bottleneck** (collo di bottiglia). La porzione più profonda della rete è dove il campo recettivo è più ampio e dove il modello può combinare le informazioni contestuali più vaste. Nei problemi inversi, questo è spesso il luogo in cui la rete risolve ambiguità che non possono essere gestite dai soli filtri locali. Per questo motivo, anche cambiamenti relativamente piccoli nel design del bottleneck possono influenzare significativamente le prestazioni.

La forza pratica della U-Net deriva dalla combinazione di tre ingredienti:

1. un campo recettivo in rapido aumento attraverso il sottocampionamento (*downsampling*);
2. la conservazione dei dettagli spaziali fini attraverso le *skip connections*;
3. un efficiente raffinamento locale attraverso le convoluzioni nel decoder.

UNet Variants

- **Residual UNet**

  Invece di costringere ogni blocco ad apprendere una trasformazione completa da zero, gli permettiamo di apprendere una correzione rispetto a una mappa identità (*identity map*). Questo spesso facilita l'ottimizzazione e aiuta i gradienti a propagarsi attraverso reti più profonde.

  Questa variante è particolarmente interessante quando:

  - la rete diventa più profonda e difficile da addestrare;
  - il *task* (compito) è naturalmente formulato come rimozione o correzione di artefatti;
  - si desidera che la rete preservi le caratteristiche (*features*) stabili, perfezionando solo ciò che deve essere cambiato.

  In pratica, i blocchi residui spesso rendono l'addestramento più robusto e possono migliorare la velocità di convergenza. Il prezzo da pagare è un design del blocco leggermente più strutturato, ma concettualmente l'architettura rimane vicina alla U-Net di base.

- **Attention UNet**

  L'idea principale è modulare le *skip connections*: invece di inoltrare inalterate tutte le caratteristiche (*features*) dell'encoder, la rete apprende dei "cancelli" (*gates*) che enfatizzano le caratteristiche più rilevanti per lo stato attuale del decoder.

  La motivazione qui è diversa dal caso residuale. Il problema non è più principalmente l'ottimizzazione, ma la **selezione delle caratteristiche** (*feature selection*). In una U-Net standard, ogni *skip connection* inoltra una grande quantità di informazioni. Questo è spesso utile, ma può anche passare al decoder attivazioni irrilevanti o debolmente informative. I cancelli di attenzione (*attention gates*) cercano di filtrare queste caratteristiche ed evidenziare le strutture spaziali che contano di più per la ricostruzione.

  Questo può essere particolarmente utile quando:

  - l'immagine contiene molte strutture ma solo alcune sono rilevanti per il compito di ricostruzione;
  - la corruzione (il degrado) è spazialmente non uniforme;
  - si desidera che il decoder si concentri maggiormente sulle regioni salienti piuttosto che trattare tutte le caratteristiche saltate (*skip features*) allo stesso modo.

- **UNet++**

  La sua motivazione è leggermente diversa: invece di modificare i blocchi convoluzionali o i cancelli delle *skip connections* (come nell'Attention U-Net), riprogetta le *skip connections* stesse. I percorsi di salto annidati (*nested skip pathways*) cercano di ridurre il **divario semantico** (*semantic gap*) tra le caratteristiche dell'encoder e quelle del decoder, perfezionandole progressivamente prima della fusione.

  In una U-Net classica, le caratteristiche (*features*) dell'encoder e del decoder alla stessa risoluzione vengono fuse direttamente, anche se possono rappresentare informazioni a stadi di astrazione piuttosto diversi. La U-Net++ affronta questa discrepanza inserendo dei raffinamenti convoluzionali intermedi lungo i percorsi di salto. In questo modo, il decoder riceve caratteristiche che sono già state parzialmente adattate e "digerite" prima della concatenazione finale.

  Questa variante è interessante quando:

  - la fusione diretta encoder-decoder sembra troppo brusca;
  - si desidera un'aggregazione di caratteristiche multi-scala più ricca;
  - si è disposti a pagare un costo computazionale e di implementazione più elevato in cambio di un allineamento potenzialmente migliore delle caratteristiche.

Vision Transformers and Loss Design

Vision Transformers (ViT)

L'idea chiave alla base del **Vision Transformer (ViT)** è semplice: invece di elaborare l'immagine pixel per pixel o mappa di caratteristiche per mappa di caratteristiche (*feature map*), per prima cosa dividiamo l'immagine in una sequenza di *patch* (piccoli riquadri o tasselli), e poi elaboriamo quella sequenza con un encoder Transformer.

Per utilizzare un ViT in un task *image-to-image* (da immagine a immagine), la strategia più comune è dividere il modello in due parti:

- un **encoder**, che trasforma l'immagine di input in una sequenza di *patch token* contestualizzati;
- un **decoder**, che mappa nuovamente quei token nel dominio dell'immagine.

Una volta che l'immagine è stata convertita in una sequenza di *token* (patch), la sequenza viene elaborata da una pila di blocchi Transformer. L'operazione centrale in ogni blocco è il meccanismo di **self-attention** (auto-attenzione).

Quindi, a ogni *token* (patch) è permesso interagire con **tutti gli altri** *token*. In questo modo, il modello può catturare dipendenze a lungo raggio (*long-range dependencies*) in modo molto più diretto rispetto a una CNN standard.

In pratica, si utilizza solitamente la **multi-head attention** (attenzione a più teste), il che significa che diverse mappe di attenzione vengono calcolate in parallelo e poi combinate. Ogni blocco Transformer contiene inoltre:

- una connessione residua (*residual connection*) attorno al layer di *self-attention*;
- un layer di normalizzazione, solitamente *LayerNorm*;
- un percettrone multistrato *feed-forward* (MLP) applicato in modo indipendente a ciascun token;
- una seconda connessione residua attorno a quella parte *feed-forward*.

Per i problemi inversi, l'attrattiva principale delle architetture ViT è che possono modellare le **interazioni globali** in modo molto naturale. Questo è interessante ogni volta che l'artefatto non è puramente locale. Ad esempio, in compiti in cui il processo di misurazione distribuisce le informazioni sull'intera immagine (come nella TAC o nella risonanza magnetica sottocampionata), o dove la coerenza a lungo raggio è cruciale, un encoder Transformer può catturare interazioni che una piccola CNN richiederebbe molti layer per rappresentare.

Allo stesso tempo, i ViT presentano anche chiari svantaggi:

- richiedono solitamente **più dati** rispetto alle CNN perché hanno pregiudizi induttivi (*inductive biases*) integrati più deboli;
- la divisione in patch introduce una **discretizzazione grossolana** dell'immagine, che può rendere più difficile il recupero dei dettagli fini;
- il costo computazionale dell'attenzione cresce in modo **quadratico** con il numero di token.

Per questi motivi, i ViT sono spesso più efficaci o in regimi con enormi quantità di dati, oppure in **architetture ibride** in cui le convoluzioni vengono utilizzate insieme all'attenzione.

Una scelta di progettazione particolarmente importante è la **dimensione della patch P**.

- Se P è **piccola**, il numero di token aumenta, il modello conserva dettagli spaziali più fini e il decoder riceve una rappresentazione più ricca dell'immagine. Tuttavia, il costo dell'attenzione cresce in modo significativo perché dipende dal quadrato del numero di token.
- Se P è **grande**, il modello diventa più economico e facile da addestrare, ma ogni token riassume una porzione più ampia dell'immagine e i dettagli fini possono andare persi più facilmente.

> ❗ Un avvertimento metodologico finale riguarda i metodi puramente **end-to-end** in generale. Anche quando producono ricostruzioni visivamente impressionanti, non impongono esplicitamente la **coerenza dei dati** (*data consistency*) con il modello di misurazione. Di conseguenza, il loro comportamento può deteriorarsi in modo significativo quando il livello di rumore cambia, quando l'operatore di acquisizione è leggermente disallineato, o quando le immagini di test differiscono dalla distribuzione di addestramento.

Metrics for Image Quality Evaluation

- **PSNR**

  Una delle metriche più classiche è il **Peak Signal-to-Noise Ratio (PSNR)** (Rapporto Segnale-Rumore di Picco). Deriva dall'errore quadratico medio (MSE) ed è solitamente definito come:

  $$\mathrm{PSNR}(\boldsymbol{x}_{true}, \boldsymbol{x}_{pred}) = 10 \log_{10} \left(\frac{\mathrm{MAX}^2}{\mathrm{MSE}(\boldsymbol{x}_{true}, \boldsymbol{x}_{pred})} \right)$$

  dove MAX è il valore massimo ammissibile per un pixel. Il PSNR si esprime in decibel tra 0 e 100 e un valore maggiore corrisponde a un errore pixel per pixel inferiore (valori accettabili si attestano tra 30 e 40 dB).

  Il PSNR è facile da calcolare ed è ancora ampiamente utilizzato, specialmente nei benchmark di *denoising* (riduzione del rumore), *deblurring* (riduzione della sfocatura) e *super-risoluzione*. Tuttavia, il suo limite principale risiede proprio nella sua origine: essendo direttamente legato all'MSE, rimane una **misura di fedeltà pixel per pixel** (*pixel-wise fidelity measure*). Pertanto, potrebbe non riuscire a riflettere ciò che gli esseri umani percepiscono effettivamente come una buona ricostruzione.

- **SSIM**

  Una seconda metrica ampiamente utilizzata è lo **Structural Similarity Index Measure (SSIM)**. A differenza del PSNR, lo SSIM non agisce puramente pixel per pixel. Confronta *patch* (riquadri) locali di immagini in termini di luminanza, contrasto e struttura, ed è quindi molto più sensibile alle distorsioni strutturali.

  Lo SSIM assume solitamente valori compresi tra 0 e 1, dove i valori più vicini a 1 indicano una maggiore somiglianza. In pratica, lo SSIM è spesso correlato meglio al giudizio visivo umano rispetto al PSNR, specialmente quando si è interessati alla conservazione strutturale.

- **LPIPS**

  Una metrica più moderna è la **LPIPS (Learned Perceptual Image Patch Similarity)**. L'idea è di confrontare due immagini non direttamente nello spazio dei pixel, ma nello spazio delle caratteristiche (*feature space*) di una rete neurale profonda. Se due immagini attivano caratteristiche interne simili in un modello pre-addestrato, allora la LPIPS le considera percettivamente vicine. Questo di solito rende la LPIPS molto più sensibile al realismo percettivo rispetto al PSNR o allo SSIM.

  Tuttavia, anche la LPIPS deve essere interpretata con cautela:

  - dipende dalla rete pre-addestrata utilizzata per l'estrazione delle caratteristiche;
  - è più costosa da calcolare rispetto a PSNR o SSIM;
  - in domini specializzati come l'imaging medico, le caratteristiche apprese su dataset di immagini naturali potrebbero non essere perfettamente allineate con la nozione di qualità rilevante in quel contesto.

Guiding the Model via Loss Function Selection

- ℓ_{L1} **(L1 Loss / MAE):** La perdita ℓ_{L1}, nota anche come Errore Assoluto Medio (MAE - *Mean Absolute Error*), è la controparte con norma ℓ_1 della MSELoss, che utilizza la norma ℓ_2.

  Sebbene la ℓ_{L1} sia spesso associata alla produzione di immagini più nitide rispetto all'MSELoss, presenta lo svantaggio teorico di non essere differenziabile nello zero. Nella pratica, i metodi basati sul sottogradiente

(*subgradient methods*) sono efficaci, ma questo può talvolta portare a un addestramento meno stabile rispetto all'MSE. Inoltre, soffre ancora di limitazioni simili a quelle dell'MSELoss, poiché rimane una metrica calcolata pixel per pixel (*pixel-wise metric*) e potrebbe quindi non riuscire a catturare il contesto strutturale.

- $\ell_{Fourier}$ **(Fourier Loss / Perdita di Fourier):** La perdita $\ell_{Fourier}$ sfrutta l'osservazione che le alte frequenze in un'immagine, che rappresentano dettagli fini e *texture*, sono spesso più difficili da ricostruire accuratamente per le reti neurali basate su convoluzione rispetto alle basse frequenze, che rappresentano l'intensità complessiva e le strutture di grandi dimensioni. Inoltre, il recupero accurato di questi dettagli fini fa spesso apparire l'immagine ricostruita più nitida e realistica all'occhio umano.

  Pertanto, può essere considerata meno strettamente locale rispetto alle perdite calcolate pixel per pixel nel dominio dell'immagine e potenzialmente più vicina a come gli esseri umani percepiscono determinati aspetti della qualità dell'immagine rispetto alla MSELoss o alla ℓ_{L1}. Tuttavia, uno svantaggio significativo è che concentrarsi principalmente sulle alte frequenze trascura le informazioni a bassa frequenza. Pertanto, la $\ell_{Fourier}$ è tipicamente utilizzata *in combinazione* con altre perdite, come MSE o ℓ_{L1} , per garantire che anche la struttura complessiva e il contesto dell'immagine vengano ricostruiti in modo accurato.

- ℓ_{SSIM} **(SSIM Loss / Perdita SSIM):** Un approccio alternativo che si concentra sulla somiglianza percettiva è la ℓ_{SSIM}. Si basa sulla misura dell'indice di somiglianza strutturale (SSIM - *Structural Similarity Index Measure*), una metrica ampiamente utilizzata e progettata per quantificare la *somiglianza visiva* tra due immagini. Lo SSIM è noto per la sua buona correlazione con la percezione umana della qualità dell'immagine perché confronta *patch* (riquadri) locali in base a luminanza, contrasto e struttura.

  La perdita ℓ_{SSIM} è molto popolare nei *task* di elaborazione delle immagini perché la sua ottimizzazione porta spesso a risultati visivamente gradevoli. Questo successo è in parte attribuito al meccanismo dello SSIM, che analizza le immagini in termini di struttura locale ed è quindi intrinsecamente più sensibile al contesto rispetto alle metriche puramente calcolate pixel per pixel (*pixel-wise* come MSE o MAE).

- $\ell_{Perceptual}$ **(Perceptual Loss / Perdita Percettiva):** Basandosi sull'idea di utilizzare metriche rilevanti per la percezione umana, le Perdite Percettive offrono un'altra potente alternativa. Invece di confrontare direttamente i pixel o di fare affidamento su metriche realizzate a mano come lo SSIM, questo approccio sfrutta le caratteristiche (*features*) estratte da una rete neurale profonda, tipicamente una pre-addestrata su un grande dataset come ImageNet (ad esempio, VGG o ResNet). L'idea centrale è che gli strati intermedi di queste reti apprendono **rappresentazioni gerarchiche** di caratteristiche visive, come bordi, texture e parti di oggetti, che sono rilevanti per la percezione. La perdita $\ell_{Perceptual}$ viene calcolata confrontando queste rappresentazioni di caratteristiche per le immagini predette e quelle reali.

  La perdita $\ell_{Perceptual}$ produce spesso ricostruzioni considerate altamente realistiche e visivamente dettagliate, eccellendo nel catturare texture e strutture complesse. Tuttavia, presenta diversi svantaggi:

  - È **computazionalmente più costosa** rispetto ad altre perdite perché richiede un passaggio in avanti (*forward pass*) aggiuntivo attraverso la rete estrattore di caratteristiche.
  - Introduce una dipendenza dallo specifico modello pre-addestrato e dai layer scelti, che potrebbero richiedere una calibrazione (*tuning*) per prestazioni ottimali.
  - La sua dipendenza da reti pre-addestrate di grandi dimensioni può renderla poco pratica per vincoli di memoria o computazionali.
  - **Il disallineamento del dominio (Domain Gap):** Le caratteristiche apprese su dataset generali come ImageNet potrebbero non essere ottimali in domini specializzati come l'imaging medico (es. Tomografia Computerizzata - TC), dove le caratteristiche specifiche del dominio sono cruciali e l'uso di modelli generici potrebbe risultare inappropriato o meno efficace.

Dato che ciascuna delle funzioni di perdita menzionate in precedenza, ovvero $\ell_{L1}, \ell_{Fourier}, \ell_{SSIM}, \ell_{Perceptual}$, così come l'MSE, presenta vantaggi e svantaggi distinti, una strategia comune e spesso altamente efficace nella pratica è l'utilizzo di una **Combination Loss** (Perdita Combinata). Questo approccio combina più funzioni di perdita in un

singolo obiettivo, tipicamente come somma ponderata. Ad esempio, una perdita combinata $\ell_{Combined}$ potrebbe essere formulata come:

$$\ell_{Combined} = \lambda_1 \ell_{Loss1} + \lambda_2 \ell_{Loss2} + \cdots + \lambda_K \ell_{LossK}$$

dove ℓ_{LossK} rappresenta una delle singole funzioni di perdita discusse in precedenza, come MSE, ℓ_{L1}, ℓ_{SSIM} o $\ell_{Perceptual}$, e $\lambda_k \geq 0$ sono iperparametri che rappresentano il peso assegnato a ciascuna componente della perdita.

Final Comparison

A questo punto, è utile riassumere il ruolo delle principali architetture *end-to-end* discusse in questo capitolo.

- **CNN:** la scelta più semplice. È locale, efficiente, facile da addestrare e fornisce un'ottima linea di base (*baseline*) quando l'artefatto è relativamente semplice.
- **ResCNN:** un naturale miglioramento rispetto a una semplice CNN quando il task può essere interpretato come rimozione di artefatti. Apprendendo il residuo, il modello spesso generalizza meglio e ricostruisce l'immagine pulita più facilmente.
- **UNet:** la scelta predefinita più forte per molti task di ricostruzione delle immagini. La sua struttura multi-scala combina in modo efficace i dettagli locali e il contesto globale, motivo per cui è così ampiamente utilizzata nella pratica.
- **ViT:** molto utile quando la struttura globale ha una forte importanza e sono disponibili dati di addestramento sufficienti. È più flessibile nel modellare le interazioni a lungo raggio, ma di solito è computazionalmente più costoso e meno efficiente dal punto di vista dei dati (*data-efficient*) rispetto ai modelli convoluzionali.

In sintesi, se si desidera una prima architettura affidabile, la **U-Net** è di solito la migliore scelta predefinita. Una **CNN** semplice è un'eccellente *baseline*, la **ResCNN** è un'opzione naturale per corruzioni di tipo residuo, e il **ViT** diventa particolarmente attraente quando le dipendenze a lungo raggio sono importanti e il dataset è abbastanza grande da supportare un addestramento basato sui Transformer.

Cross-Domain End-to-End Reconstruction

di input (y) e l'immagine target (x) vivano sulla stessa griglia spaziale. Questo è il caso, ad esempio, del *denoising* (riduzione del rumore) e del *deblurring* (riduzione della sfocatura): l'immagine corrotta e la ricostruzione sono entrambe immagini bidimensionali standard, quindi una CNN o una U-Net possono elaborare l'input direttamente.

Molti problemi inversi non hanno questa struttura. Nella **Tomografia Computerizzata (TC)**, per esempio, la misurazione è un **sinogramma**, mentre l'output desiderato è un'immagine anatomica. L'input e l'output hanno quindi dimensioni diverse, geometria diversa e semantica diversa. Questa è l'impostazione tipica di un **problema inverso a dominio incrociato** (*cross-domain inverse problem*).

Una soluzione allettante è ridimensionare il sinogramma in modo che abbia la stessa forma dell'immagine target e poi darlo in pasto a una CNN o a una U-Net. Questo può rendere le forme dei tensori compatibili, ma **non** risolve il vero problema. Il ridimensionamento (*resizing*) cambia la forma dell'array, ma non ripristina il corretto significato spaziale dei dati.

Per questo motivo, la ricostruzione a dominio incrociato richiede solitamente un primo passo che mappi nuovamente la misurazione nel dominio dell'immagine prima che venga applicata la rete neurale.

Preprocessing via Filtered Backprojection

Una soluzione classica consiste nell'introdurre un operatore di **pre-elaborazione** veloce che mappa la misurazione rumorosa y^δ in una stima grossolana nel dominio dell'immagine $\tilde{x}$. Nella TC (Tomografia Computerizzata), la scelta standard è la **Retroproiezione Filtrata (FBP - *Filtered Backprojection*).**

Il ruolo di questo passaggio non è risolvere perfettamente il problema inverso. Il suo scopo principale è colmare il divario di dominio (*domain gap*). Una volta che i dati sono stati mappati nuovamente nel dominio dell'immagine, una CNN o una U-Net può operare su un oggetto con la geometria e l'interpretazione spaziale corrette.

Questo porta alla pipeline ibrida:

$$y^{\delta} \xrightarrow{\text{FBP}} \tilde{x} \xrightarrow{f_{\Theta}} x_{\text{pred}}$$

Partendo da un'immagine pulita x, generiamo prima un sinogramma $y = Kx$, poi aggiungiamo rumore per ottenere y^{δ}, e infine calcoliamo la ricostruzione FBP $\tilde{x} = \text{FBP}(y^{\delta})$.

Il punto chiave è che alla rete neurale **non** viene chiesto di invertire direttamente l'operatore tomografico. Impara invece a migliorare una ricostruzione grossolana che vive già nel dominio corretto.

Deep Generative Models, VAEs, and GANs

Finora, il corso si è concentrato principalmente sui **modelli di ricostruzione**: dato un dato misurato $\boldsymbol{y}^{\delta}$, addestrare una rete neurale che produca una stima dell'immagine $\boldsymbol{x}_{\text{pred}}$. Questo è un punto di vista **discriminativo**, perché il modello apprende una mappa diretta da una variabile all'altra.

Un **modello generativo profondo** (*deep generative model*) ha un obiettivo diverso. Invece di prevedere una quantità da un'altra, cerca di apprendere la distribuzione delle immagini stesse. Se la distribuzione dei dati è denotata da $p_{\text{data}}(\boldsymbol{x})$, l'obiettivo è costruire un modello parametrizzato che la approssimi abbastanza bene da generare nuovi campioni realistici, catturare la variabilità del dataset e fornire un *prior* (una conoscenza a priori) appreso per i problemi inversi.

Deep Generative Models and Deep Latent Variable Models

Al livello più generale, un modello generativo è un modello della legge di probabilità dei dati. Se le immagini sono denotate da $\boldsymbol{x} \in \mathbb{R}^n$, l'obiettivo ideale è la distribuzione sconosciuta $p_{\text{data}}(\boldsymbol{x})$.

Un modello generativo profondo introduce parametri addestrabili Θ e definisce una famiglia di modelli

$$p_{\Theta}(\boldsymbol{x})$$

che viene addestrata affinché p_{Θ} diventi vicina a p_{data}. Una strategia molto comune consiste nell'introdurre una **variabile latente** a dimensione inferiore $\boldsymbol{z} \in \mathbb{R}^d$, con $d \ll n$, e nel descrivere la generazione di immagini in due fasi:

$$\boldsymbol{z} \sim p(\boldsymbol{z}), \qquad \boldsymbol{x} \sim p_{\Theta}(\boldsymbol{x} \mid \boldsymbol{z})$$

Questa è la struttura generale di un **Deep Latent Variable Model (DLVM)** (Modello a Variabile Latente Profondo). Il *prior* (probabilità a priori) $p(\boldsymbol{z})$ è solitamente scelto per essere semplice, ad esempio una Gaussiana standard $\mathcal{N}(0, I)$, mentre la legge condizionale $p_{\Theta}(\boldsymbol{x} \mid \boldsymbol{z})$ è resa espressiva da una rete neurale.

Un vettore latente deve essere pensato come una descrizione interna compressa dell'immagine. Non è destinato a memorizzare ogni pixel uno ad uno. Codifica invece i fattori principali che spiegano l'immagine in una forma più compatta.

In altre parole, il vettore latente non è solitamente interpretabile coordinata per coordinata, ma è comunque utile pensarlo come una **rappresentazione compressa dell'oggetto**. Il decodificatore (*decoder*) prende quindi questa descrizione compressa e la espande nuovamente in un'immagine completa. Questo è il motivo per cui i modelli a variabili latenti sono così attraenti: suggeriscono che immagini altamente strutturate possano vivere vicino a una **varietà (*manifold*)** di dimensione molto inferiore all'interno dello spazio dei pixel circostante.

- In un **modello generativo esplicito**, si scrive direttamente un modello probabilistico e si cerca di ottimizzare o approssimare una *likelihood* (verosimiglianza). Questo è il punto di vista alla base dei **VAE (Variational Autoencoders)**.
- In un **modello generativo implicito**, si definisce un meccanismo di campionamento

$$\boldsymbol{x} = G_\Theta(\boldsymbol{z}), \qquad \boldsymbol{z} \sim p(\boldsymbol{z})$$

  senza necessariamente avere una formula trattabile per $p_\Theta(\boldsymbol{x})$. Questo è il punto di vista alla base delle **GAN** (Generative Adversarial Networks - *Reti Avversarie Generative*).

La parola **generatore** ha quindi un significato preciso: è la mappa che trasforma una semplice variabile casuale $\boldsymbol{z}$ in un campione complesso $\boldsymbol{x}$ che dovrebbe assomigliare ai dati di addestramento.

Typical Neural Architectures for Image Generators

La maggior parte dei generatori di immagini utilizzati in pratica è costruita a partire da **blocchi codificatore-decodificatore convoluzionali** (*convolutional encoder-decoder blocks*).

- Un **encoder** (codificatore) riduce progressivamente la risoluzione spaziale aumentando al contempo il numero di canali. In questo modo, il contenuto locale dell'immagine viene compresso in mappe di caratteristiche (*feature maps*) più astratte e infine in una rappresentazione latente compatta.
- Un **decoder** (decodificatore) fa l'opposto: parte da un codice latente o da un tensore di caratteristiche grossolano e lo campiona verso l'alto (*upsampling*) riportandolo progressivamente alle dimensioni dell'immagine.
- Un **bottleneck** (collo di bottiglia) si trova al centro e costringe il modello a riassumere l'immagine attraverso una rappresentazione latente compatta.

L'architettura precisa dipende dalla famiglia di modelli.

- In un **VAE**, di solito si ha una struttura *encoder-decoder* che è molto vicina a un AutoEncoder.
- In una **GAN**, il generatore è solitamente una pura rete di tipo *decoder*, mentre il discriminatore è una rete di tipo *encoder* che mappa un'immagine in un punteggio scalare (*scalar score*).

> ! Un problema pratico molto comune nei generatori di immagini è la comparsa di **artefatti a scacchiera** (*checkerboard artifacts*). Questi artefatti sono spesso causati dalle *transposed convolutions* (convoluzioni trasposte), poiché il campionamento verso l'alto (*upsampling*) appreso può produrre pattern di sovrapposizione irregolari sulla griglia dei pixel. Quando ciò accade, un'alternativa più sicura è separare le due operazioni: prima eseguire l'upsampling della *feature map* esplicitamente, e solo in seguito applicare una convoluzione standard.

Variational Autoencoders (VAE)

Il codificatore (*encoder*) E_ϕ comprime l'immagine di input in un codice latente, mentre il decodificatore (*decoder*) D_Θ ricostruisce l'immagine a partire da quel codice.

Per le immagini, questa architettura è solitamente convoluzionale: l'encoder riduce progressivamente la risoluzione dell'immagine e aumenta il numero di canali, mentre il decoder rispecchia questo processo e aumenta la risoluzione (*upsamples*) tornando alle dimensioni originali. Questa è esattamente la struttura architettonica di base riutilizzata dalla maggior parte dei VAE.

Il VAE mantiene questa logica encoder-decoder, ma cambia il collo di bottiglia (*bottleneck*) latente in modo cruciale. Invece di mappare l'immagine su un singolo codice deterministico $\boldsymbol{z}$, l'encoder produce i parametri di una distribuzione di probabilità.

La quantità che si vorrebbe idealmente massimizzare è $\log p_\Theta(\boldsymbol{x})$, ma poiché questo è difficile, il VAE ottimizza l'**Evidence Lower Bound (ELBO)** (Limite Inferiore dell'Evidenza):

$$\log p_\Theta(\boldsymbol{x}) \geq \mathbb{E}_{q_\phi(\boldsymbol{z}|\boldsymbol{x})}\big[\log p_\Theta(\boldsymbol{x} \mid \boldsymbol{z})\big] - \mathrm{KL}\big(q_\phi(\boldsymbol{z} \mid \boldsymbol{x}) \,\|\, p(\boldsymbol{z})\big)$$

Questo obiettivo ha un'interpretazione molto naturale:

- Il **termine di ricostruzione** forza il decoder a preservare abbastanza informazioni per ricostruire l'immagine.
- Il **termine KL** regolarizza la rappresentazione latente in modo che rimanga vicina al semplice *prior* gaussiano.

L'encoder parte da un blocco convoluzionale superficiale (*stem*) e poi applica quattro blocchi di sottocampionamento (*downsampling*). Ad ogni stadio, la dimensione spaziale viene dimezzata,

$$64 \times 64 \to 32 \times 32 \to 16 \times 16 \to 8 \times 8 \to 4 \times 4$$

mentre il numero di canali aumenta come

$$1 \to 32 \to 64 \to 128 \to 256 \to 512$$

Ogni blocco contiene una convoluzione, una normalizzazione, una non-linearità e un affinamento residuo (*residual refinement*). Il tensore finale di dimensione 512×4×4 viene appiattito (*flattened*) e mappato in due vettori, ovvero la media $\boldsymbol{\mu}$ e la log-varianza $\log \boldsymbol{\sigma}^2$, ciascuno di dimensione 64. Questo è il collo di bottiglia latente del modello.

Il decoder rispecchia la stessa logica multi-scala. Partendo da un vettore latente $\boldsymbol{z} \in \mathbb{R}^{64}$, un layer lineare lo rimodella (*reshapes*) inizialmente in un tensore di feature grossolano di dimensione 512×4×4. Successivamente, quattro blocchi di sovracampionamento (*upsampling*) ricostruiscono progressivamente l'immagine riportandola alla dimensione 64×64.

In pratica, i VAE sono attraenti perché forniscono uno spazio latente ben strutturato e un obiettivo probabilistico rigoroso (*principled*). Tuttavia, bisogna essere precisi su cosa può andare storto. Un VAE potrebbe ricostruire le immagini abbastanza bene ma generare comunque pessimi campioni casuali. Questo accade quando il decoder impara a lavorare bene sui vettori latenti codificati prodotti dalle immagini di addestramento, ma quei vettori latenti non sono sufficientemente vicini alla distribuzione a priori $p(\boldsymbol{z}) = \mathcal{N}(0, I)$. In tal caso, il modello si comporta più come un forte AutoEncoder che come un vero modello generativo.

prior fissato esplicitamente con la LIKELYHOOD

> ❗ Se il termine KL è troppo debole, il modello potrebbe ricostruire bene ma fallire come modello generativo, perché i vettori latenti codificati si allontanano dal *prior* gaussiano. Se il termine KL è troppo forte troppo presto, il modello potrebbe invece collassare producendo ricostruzioni sfocate. Nella pratica, è necessario un compromesso: un collo di bottiglia che sia sufficientemente espressivo per la ricostruzione, ma sufficientemente regolarizzato affinché il campionamento dal *prior* rimanga significativo.

Generative Adversarial Networks (GAN)

Una **Generative Adversarial Network (GAN)** intraprende una strada molto diversa. Invece di massimizzare una verosimiglianza (*likelihood*) esplicita, introduce due reti neurali che giocano un gioco l'una contro l'altra.

- Il **generatore** è una mappa neurale $G_\Theta : \mathbb{R}^d \to \mathbb{R}^n$, che trasforma un codice latente $\boldsymbol{z} \sim p(\boldsymbol{z})$ in un'immagine sintetica $\boldsymbol{x}_{\text{fake}} = G_\Theta(\boldsymbol{z})$.
- Il **discriminatore** o **critico** è un'altra rete neurale, denotata da D_Ψ, che riceve un'immagine e produce un punteggio indicante se l'immagine sembra reale o falsa.

La variabile latente $\boldsymbol{z}$ viene data in pasto al generatore G_Θ, che produce un'immagine candidata, mentre il discriminatore D_Ψ riceve o un'immagine reale $\boldsymbol{x}_{\text{real}}$ o una generata $\boldsymbol{x}_{\text{fake}}$ e restituisce un punteggio scalare.

Da un punto di vista architettonico, le GAN per le immagini sono ancora una volta solitamente convoluzionali, ma la logica è diversa rispetto al caso dei VAE.

- Il **generatore** è tipicamente un'architettura di tipo *decoder*: parte da un vettore latente, lo espande in un tensore di caratteristiche (*feature tensor*) grossolano e aumenta progressivamente la risoluzione (*upsampling*) fino a raggiungere la dimensione dell'immagine.
- Il **discriminatore** è tipicamente un'architettura di tipo *encoder*: riduce progressivamente la risoluzione dell'immagine (*downsampling*), aumenta il numero di canali delle caratteristiche e infine mappa il risultato in un punteggio scalare.

Il generatore parte da un vettore latente $\boldsymbol{z} \in \mathbb{R}^{128}$. Un layer lineare mappa prima questo vettore in un tensore grossolano di dimensione $512 \times 4 \times 4$. Da lì, quattro blocchi di upsampling residuo ricostruiscono l'immagine progressivamente:

$$4 \times 4 \to 8 \times 8 \to 16 \times 16 \to 32 \times 32 \to 64 \times 64$$

mentre le dimensioni dei canali diminuiscono come:

$$512 \to 256 \to 128 \to 64 \to 32 \to 1$$

Il **critico** segue la direzione opposta. Riceve un'immagine 64×64 e applica una sequenza di blocchi residui di sottocampionamento (*downsampling*) finché la risoluzione non è ridotta a 4×4 e il numero di canali raggiunge 512. Il risultato viene appiattito (*flattened*) e mappato in un singolo punteggio scalare.

Le GAN sono diventate famose perché possono produrre immagini visivamente nitide. Allo stesso tempo, sono notoriamente delicate da addestrare. L'ottimizzazione è un gioco a due giocatori, il discriminatore può diventare troppo forte o troppo debole, e il generatore può soffrire di **mode collapse** (collasso delle modalità), producendo solo un piccolo sottoinsieme dei possibili output.

Generative Priors for Inverse Problems

Consideriamo una misurazione sfocata e rumorosa:

$$\boldsymbol{y}^{\delta} = K\boldsymbol{x}^{\dagger} + \boldsymbol{e}$$

Se si assume che l'immagine ignota si trovi approssimativamente nel *range* (campo d'azione) di un generatore G, allora si può cercare un codice latente la cui immagine generata spieghi i dati:

$$\widehat{\boldsymbol{z}} = \arg\min_{\boldsymbol{z}} \|KG(\boldsymbol{z}) - \boldsymbol{y}^{\delta}\|_2^2 + \lambda\|\boldsymbol{z}\|_2^2$$

La ricostruzione è quindi:

$$\widehat{\boldsymbol{x}} = G(\widehat{\boldsymbol{z}})$$

Da un punto di vista bayesiano, questo è uno stimatore **MAP** (*Maximum A Posteriori*) a variabili latenti. Si parte da una semplice legge latente, spesso $\boldsymbol{z} \sim \mathcal{N}(\mathbf{0}, I)$, la si spinge attraverso il generatore e in questo modo si induce un *prior* sulle immagini. Nel caso del VAE, il decoder gioca il ruolo di G e il *prior* latente è integrato nel modello. Nel caso della GAN, il generatore è esplicito, ma la densità dell'immagine indotta è solitamente implicita piuttosto che analiticamente trattabile.

- Il **prior del VAE** è spesso più facile da ottimizzare perché il suo spazio latente è più regolare e fluido.
- Il **prior della GAN** può produrre risultati più nitidi, ma l'ottimizzazione latente può essere più fragile.

L'ottimizzazione latente è **non convessa**, può dipendere fortemente dall'inizializzazione e dagli iperparametri, e può barattare la coerenza dei dati con il realismo. I *prior* delle GAN sono particolarmente sensibili al *mode collapse* o a una

geometria latente irregolare; i *prior* dei VAE sono solitamente più stabili ma possono essere più sfocati.

Il costo principale è il **bias (pregiudizio) del generatore**. Se l'immagine reale si trova al di fuori o lontano dal raggio d'azione del generatore, il metodo può recuperare solo l'immagine plausibile più vicina secondo il modello, non necessariamente quella corretta.

> ❗ Una ricostruzione visivamente plausibile ottenuta da un **Deep Generative Prior (DGP)** non è automaticamente una ricostruzione affidabile. Nei metodi DGP, il *prior* è imposto dal raggio d'azione del generatore stesso; pertanto, la mancanza di modalità (*missing modes*), i pregiudizi di addestramento (*training bias*) o gli errori di ottimizzazione latente possono apparire direttamente sotto forma di strutture allucinate o dettagli soppressi. Questa limitazione è una delle ragioni principali per cui i **prior basati sulla diffusione (diffusion-based priors)** sono diventati così importanti: essi sono solitamente molto più ricchi e flessibili rispetto a un generatore latente fisso a bassa dimensione.

Diffusion Models: DDPM and DDIM

I VAE e le GAN sono storicamente fondamentali, ma la modellazione generativa moderna è dominata da una terza famiglia: i **modelli di diffusione**. Questi modelli generano dati imparando come invertire un processo graduale di aggiunta di rumore (*noising process*). In pratica, hanno riscosso un enorme successo perché combinano un'elevata qualità dei campioni, una relativa stabilità nell'addestramento e una chiara interpretazione probabilistica.

The Forward Diffusion Process

L'idea centrale è definire una sequenza di variabili latenti

$$\boldsymbol{x}_0, \boldsymbol{x}_1, \ldots, \boldsymbol{x}_T,$$

dove $\boldsymbol{x}_0$ è un'immagine pulita campionata dalla distribuzione dei dati e le variabili successive sono progressivamente più rumorose.

Per piccoli step temporali (*timesteps*), l'immagine è solo leggermente perturbata; per grandi step temporali, diventa quasi indistinguibile dal rumore gaussiano. Il problema di apprendimento è quindi: dato $(\boldsymbol{x}_t, t)$, prevedere abbastanza informazioni per invertire questo processo di corruzione. Nei **Denoising Diffusion Probabilistic Models (DDPM)**, il processo inverso è parametrizzato da una rete neurale che prevede la componente di rumore in $\boldsymbol{x}_t$.

$$\boldsymbol{x}_t = \sqrt{\alpha_t}\boldsymbol{x}_0 + \sqrt{1-\alpha_t}\boldsymbol{\epsilon}_t, \qquad \boldsymbol{\epsilon}_t \sim \mathcal{N}(0, I).$$

Per un istante temporale casuale t, possiamo corrompere analiticamente un'immagine pulita e chiedere alla rete neurale di prevedere il rumore che è stato utilizzato.

> 🧠 L'addestramento alla diffusione non richiede una rete per ogni livello di rumore. Una singola rete neurale condizionata dal tempo (*time-conditioned*) viene addestrata per risolvere molti problemi di denoising contemporaneamente.

Uno degli aspetti più eleganti dei DDPM è che al modello non viene chiesto di generare un'immagine direttamente. Invece, apprende una famiglia di compiti di denoising attraverso molti livelli di rumore. Dall'output della rete, è possibile ricostruire una stima dell'immagine pulita.

Denoising Architectures for Diffusion Models

In linea di principio, potrebbero essere utilizzate molte architetture *image-to-image*, ma in pratica la scelta standard è un **denoiser in stile UNet**.

La ragione è strutturale. Il denoiser deve funzionare bene a diverse scale spaziali contemporaneamente. All'inizio del processo inverso, quando t è grande, la rete deve recuperare la struttura globale grossolana da un'immagine molto rumorosa. Più tardi, quando t è piccolo, deve ripristinare i dettagli locali fini senza distruggere ciò che è già corretto. Una UNet è ben adatta a questo perché il percorso di contrazione (*contracting path*) costruisce ampi campi ricettivi e cattura il contesto globale, mentre il percorso di espansione (*expanding path*) ricostruisce i dettagli spaziali utilizzando le **skip connections** dalle caratteristiche ad alta risoluzione. In altre parole, combina semantica e localizzazione nella stessa architettura, che è esattamente ciò di cui ha bisogno il denoising per diffusione.

Invece di fornire lo scalare t direttamente alla rete, lo si mappa in un vettore le cui coordinate sono funzioni sinusoidali di diverse frequenze, ad esempio termini della forma $\sin(\omega_k t)$ e $\cos(\omega_k t)$ per diverse scale ω_k. Le componenti a bassa frequenza variano lentamente lungo il tempo di diffusione e catturano informazioni grossolane sul fatto che l'immagine sia ancora molto rumorosa o già vicina ad essere pulita, mentre le componenti ad alta frequenza permettono alla rappresentazione di distinguere gli step temporali vicini in modo più preciso.

Ad ogni iterazione carichiamo un'immagine pulita $\boldsymbol{x}_0$, campioniamo uno step temporale casuale t, campioniamo il rumore gaussiano $\boldsymbol{\epsilon}_t$, formiamo l'immagine rumorosa $\boldsymbol{x}_t$ analiticamente e quindi addestriamo la rete a prevedere $\boldsymbol{\epsilon}_t$ a partire da $(\boldsymbol{x}_t, t)$.

Sampling with DDPM and DDIM

Una volta addestrato il denoiser, il campionamento **DDPM** inizia dal rumore gaussiano puro $\boldsymbol{x}_T \sim \mathcal{N}(0, I)$ e applica la catena inversa appresa da t=T fino a t=0. Ogni passaggio combina due ingredienti: la previsione della rete $\boldsymbol{\epsilon}_\Theta(\boldsymbol{x}_t, t)$, che stima quanto rumore è presente, e un termine casuale gaussiano, che mantiene il processo stocastico. Ciò rende il campionamento DDPM fedele al modello probabilistico sottostante, ma anche relativamente lento.

I **Denoising Diffusion Implicit Models (DDIM)** introducono un processo inverso non markoviano alternativo che utilizza lo stesso denoiser addestrato ma consente una generazione molto più rapida.

Concettualmente, **DDPM** e **DDIM** condividono lo stesso denoiser ma non la stessa traiettoria generativa. DDPM segue una catena stocastica di diffusione inversa, mentre DDIM segue un percorso non markoviano che può saltare molti step temporali (*timesteps*). Questo è il motivo per cui la stessa rete addestrata può supportare sia un campionamento ad alta fedeltà ma lento, sia una generazione approssimativa molto più veloce.

> ❗ DDIM è più veloce di DDPM, ma non è semplicemente un "aumento di velocità gratuito". Esso modifica la dinamica del campionamento, quindi lo si dovrebbe considerare come un campionatore differente costruito sopra lo stesso denoiser addestrato.

dall'output del diffusion model si può ricavare l'immagine denoised

Diffusion Models for Inverse Problems

Una volta che un modello generativo è stato addestrato, può essere utilizzato come *prior* per la ricostruzione. L'idea centrale è probabilistica: invece di cercare un'immagine che si adatti solo al dato misurato, cerchiamo un'immagine che sia sia **coerente con i dati** (*data-consistent*) sia **plausibile secondo la distribuzione delle immagini appresa**.

Quindi, la ricostruzione dovrebbe combinare due pezzi di informazione: la **verosimiglianza**, che misura l'accordo con i dati, e il **prior**, che misura la plausibilità dell'immagine. → il diffusion model può essere usato come prior x generare l'immagine finale

I modelli a variabili latenti come i VAE e le GAN forniscono già un *prior* appreso, ma vincolano la soluzione al campo d'azione (*range*) di un generatore a bassa dimensione. I modelli di diffusione sono più ricchi. Invece di limitare l'immagine a trovarsi su una varietà (*manifold*) latente fissa, imparano come rimuovere il rumore da immagini

realistiche attraverso molti livelli di rumore. Ciò dà accesso a due oggetti strettamente correlati: un predittore di rumore $\boldsymbol{\epsilon}_\Theta(\boldsymbol{x}_t, t)$ e la corrispondente stima dell'immagine pulita $\hat{\boldsymbol{x}}_0(\boldsymbol{x}_t, t)$.

Vale la pena essere precisi su cosa significhi *prior* in questo contesto. Un modello di diffusione di solito **non** ci fornisce una densità $p(\boldsymbol{x})$ trattabile in forma chiusa. Ciò che ci fornisce sono informazioni di denoising, o equivalentemente informazioni sullo **score** (punteggio) per versioni rumorose della distribuzione dei dati. Questo è sufficiente per guidare la ricostruzione iterativa, perché l'algoritmo può chiedere ripetutamente: tra le immagini che si adattano alle misurazioni, in quale direzione il modello appreso ritiene che l'immagine diventi più realistica?

DPS and DiffPIR

I due metodi discussi di seguito utilizzano entrambi lo stesso *prior* di diffusione addestrato, ma lo combinano con il modello di misurazione in modi diversi.

- **DPS (Diffusion Posterior Sampling)** utilizza un gradiente di verosimiglianza (*likelihood gradient*) durante il processo di diffusione inversa.

  Lo stato $\boldsymbol{x}_t$ è l'attuale iterata rumorosa del processo inverso. Il denoiser produce quindi una stima dell'immagine pulita $\hat{\boldsymbol{x}}_0$ e, da tale stima, si può valutare quanto la ricostruzione corrisponda alle misurazioni. **DPS** utilizza tale discrepanza non come un passaggio di proiezione separato, ma come un **segnale di gradiente** che modifica la traiettoria di diffusione inversa stessa.

  In pratica, non sappiamo come valutare questa quantità esattamente in $\boldsymbol{x}_t$, perché le misurazioni vengono naturalmente confrontate con un'immagine pulita piuttosto che con uno stato latente rumoroso. Il **DPS** risolve questo problema utilizzando la stima indotta dal denoiser $\hat{\boldsymbol{x}}_0(\boldsymbol{x}_t, t)$ come *proxy* per l'immagine pulita sottostante e poi differenziando il termine di fedeltà ai dati attraverso tale stima. Ciò fornisce una direzione di guida che viene poi utilizzata per orientare il passaggio inverso.

  Simbolicamente, si può pensare all'aggiornamento come:

  $$\boldsymbol{x}_s \approx R_\Theta(\boldsymbol{x}_t, t, s) - \eta \boldsymbol{g}_t$$

  dove R_Θ denota il passaggio di diffusione inversa guidato dal *prior* e η è la forza della guida (*guidance strength*).

  Questo punto di vista ha un vantaggio principale: il DPS mantiene il *prior* di diffusione completamente nel ciclo e lo adatta al problema inverso **senza riaddestrare** il modello generativo. È anche flessibile: finché si può calcolare o approssimare un gradiente di verosimiglianza, lo stesso *prior* può essere riutilizzato per diversi operatori in avanti. Separa l'apprendimento del *prior* dall'acquisizione dei dati.

  Le limitazioni sono altrettanto importanti. In primo luogo, il metodo è computazionalmente pesante perché ogni passaggio inverso richiede sia una valutazione del denoiser che una correzione della coerenza dei dati basata sul gradiente. In secondo luogo, la forza della guida (*guidance strength*) deve essere sintonizzata con cura: se è troppo piccola, la ricostruzione può ignorare le misurazioni e deviare verso immagini visivamente plausibili ma incoerenti con i dati; se è troppo grande, il campionatore può diventare instabile o eccessivamente nitido (*oversharpened*). In terzo luogo, il gradiente di verosimiglianza è solo approssimativo, perché è instradato attraverso $\hat{\boldsymbol{x}}_0$ piuttosto che attraverso un modello posteriore esatto. Pertanto, il **DPS** deve essere inteso come un campionatore guidato dal posteriore praticamente efficace, non come una garanzia "scatola nera" di campionamento bayesiano esatto.

- **DiffPIR (Diffusion Plug and play Image Restoration)** segue una filosofia *plug-and-play* e alterna un passaggio di denoising a un passaggio di coerenza dei dati (*data-consistency*).

  Ad ogni timestep, partiamo da uno stato corrente $\boldsymbol{x}_t$, usiamo il modello di diffusione per produrre una stima guidata dal *prior* $\hat{\boldsymbol{x}}_0$, e poi applichiamo esplicitamente una mappa di coerenza dei dati:

  $$\boldsymbol{D}_t = C(\hat{\boldsymbol{x}}_0, K, \boldsymbol{y}^\delta)$$

prima di passare all'iterata successiva. In altre parole, l'informazione a priori e la coerenza con le misurazioni sono gestite in due sottopassaggi concettualmente distinti.

DiffPIR segue una filosofia **plug-and-play**. Invece di cercare di scrivere una transizione posteriore esatta, alterna due operazioni: un passaggio di denoising del modello di diffusione, che spinge l'iterata verso il *prior* d'immagine appreso, e un passaggio di coerenza dei dati, che la spinge nuovamente verso l'accordo con le misurazioni.

Questa non è altro che una correzione di discesa del gradiente per il termine quadratico di fedeltà ai dati. Il denoiser e il modello in avanti agiscono quindi in alternanza: il denoiser migliora la plausibilità e la correzione esplicita ripristina la coerenza fisica.

I vantaggi di DiffPIR sono quindi la modularità, l'interpretabilità e la vicinanza concettuale all'ottimizzazione. Spesso si può adattare il passaggio di coerenza dei dati alla struttura dell'operatore K in modo del tutto naturale, e si può ragionare sul parametro τ come su una dimensione standard del passo di ottimizzazione. DiffPIR tende inoltre a produrre ricostruzioni che rimangono visibilmente ancorate alle misurazioni, perché la coerenza dei dati viene imposta esplicitamente ad ogni iterazione.

Le limitazioni derivano dalla medesima struttura di scomposizione (*splitting*). Il metodo non viene qui derivato come un campionatore posteriore esatto, quindi non si dovrebbero interpretare automaticamente le sue iterate come campioni del vero posteriore bayesiano. Il suo comportamento dipende fortemente dal bilanciamento tra denoising e correzione: se il passaggio di denoising è troppo dominante, il risultato può allucinare dettagli plausibili ma errati; se il passaggio di coerenza dei dati è troppo forte, il metodo può diventare eccessivamente levigato (*oversmoothed*) o oscillatorio. Inoltre, la netta separazione tra i termini del *prior* e dei dati è algoritmicamente conveniente, ma potrebbe essere meno fedele alla vera geometria del posteriore rispetto a metodi che accoppiano entrambi gli ingredienti in modo più stretto.

In breve: **DPS** è più vicino all'idea di **orientare una traiettoria di diffusione posteriore**, mentre **DiffPIR** è più vicino all'**alternanza tra un denoiser appreso e una correzione in stile ottimizzazione**. Il codice sottostante enfatizza intenzionalmente questa differenza.

> ❗ Una ricostruzione basata sulla diffusione può apparire estremamente convincente anche quando non rappresenta il corretto campione posteriore, o persino quando non è pienamente coerente con i dati misurati. Questo è il motivo per cui la **valutazione quantitativa** e un'**ispezione accurata della coerenza dei dati** rimangono essenziali.